

# Phase 4

## What is SocketCAN?

In a real vehicle:

None

Engine ECU ----\

ABS ECU ----- CAN Bus ----- TCU

BCM ECU ----/

Cluster ECU -/

ECUs exchange CAN frames over the CAN bus.

Since we don't have actual CAN hardware, Linux provides a Virtual CAN Interface (VCAN):

None

vcan0

It behaves like a real CAN bus entirely in software.

---

## Phase 4 Goal

We'll simulate:

| CAN ID | Signal             |
|--------|--------------------|
| 0x100  | Vehicle Speed      |
| 0x101  | Engine RPM         |
| 0x102  | Engine Temperature |

**Your TCU Simulator will:**

None

**Generate Data**



**Send CAN Frames**



**vcan0**



**CAN Monitor / Receiver**

## **Step 1: Install CAN Utilities**

None

```
sudo apt update  
sudo apt install can-utils
```

Verify:

None

```
candump --help
```

## Step 2: Load Virtual CAN Module

None

```
sudo modprobe vcan
```

Verify:

None

```
lsmod | grep vcan
```

Expected:

None

```
vcan
```

---

## Step 3: Create Virtual CAN Interface

None

```
sudo ip link add dev vcan0 type vcan
```

Bring it up:

None

```
sudo ip link set up vcan0
```

Verify:

None

```
ip link show vcan0
```

Expected:

None

```
vcan0: <NOARP,UP,LOWER_UP>
```

---

## Step 4.1: Test CAN Manually

### Terminal 1

Listen:

None

```
candump vcan0
```

### Terminal 2

Send:

None

```
cansend vcan0 100#64
```

Expected in Terminal 1:

None

```
vcan0 100 [1] 64
```

If this works:

- ✓ SocketCAN installed
- ✓ Virtual CAN working

## Step 4.2: Test Multiple Automotive Signals

Try:

### TERMINAL 2 : Vehicle Speed

None

```
cansend vcan0 100#3C
```

(0x3C = 60 km/h)

---

### TERMINAL 3 : RPM

None

```
cansend vcan0 101#0BB8
```

(0x0BB8 = 3000 RPM)

---

## TERMINAL 4 : Engine Temperature

None

```
cansend vcan0 102#5A
```

(0x5A = 90°C)

---

Expected in candump:

TERMINAL 1 :

```
aniruddhan@raspberrypi:~ $ candump vcan0
```

None

```
vcan0 100 [1] 64
```

```
vcan0 101 [2] 0B B8
```

```
vcan0 102 [1] 5A
```

THIS proves that:

| Feature   | Status                                                                              |
|-----------|-------------------------------------------------------------------------------------|
| SocketCAN |  |



vcan0 Interface 

CAN Frame  
Transmission 

CAN Frame  
Reception 

can-utils 

Phase 4  
Environment Setup 

---

## Step 5: Integrate CAN into Your Existing TCU

We'll add:

None

```
#include <linux/can.h>
#include <linux/can/raw.h>
#include <sys/socket.h>
#include <sys/ioctl.h>
#include <net/if.h>
```

```
#include <unistd.h>
```

Create:

None

```
int can_socket;
```

Configure:

None

```
socket(PF_CAN, SOCK_RAW, CAN_RAW);
```

Bind to:

None

```
vcan0
```

Then transmit:

## Speed

None

```
CAN ID: 0x100
```

```
Data: speed
```

## RPM

None

```
CAN ID: 0x101
```

```
Data: rpm
```

## Temperature



None

CAN ID: 0x102

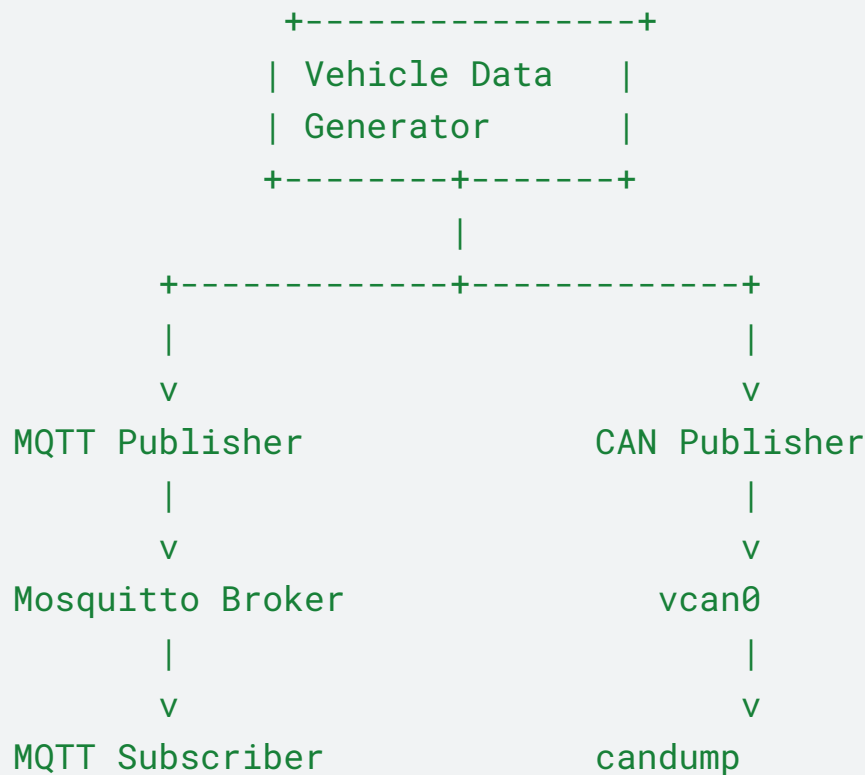
Data: engineTemp

every second along with MQTT publishing.

---

## Target Architecture After Phase 4

None



## First, complete Steps 1–4

Run:

None

```
which candump
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
ip link show vcan0
```

Then test:

None

```
candump vcan0
```

and in another terminal:

None

```
cansend vcan0 100#64
```

=====

## Current architecture:

None

```
Vehicle Data Generator
  |
+---- MQTT ----> Mosquitto
  |
+---- CAN -----> vcan0
```

Every second your program will:

None

Generate Speed

Generate RPM

Generate Temp

Generate GPS

Publish MQTT Topics

Send CAN ID 0x100 -> Speed

Send CAN ID 0x101 -> RPM

Send CAN ID 0x102 -> Engine Temp

Then:

None

candump vcan0

will show live CAN traffic while MQTT subscribers simultaneously receive messages.

---

## Why This Is Valuable

At this stage your project starts resembling a real TCU:

None

```
+-----+
| Linux TCU          |
| Raspberry Pi 4B    |
+-----+-----+
|
```

```
+----- MQTT (Telematics)
|
+----- CAN (Vehicle Network)
```

These are two core technologies used in automotive systems.

### **CODE : MQTT+CAN**

tcu\_mqttCAN.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <unistd.h>
#include <cstring>
#include <mosquitto.h>

// CAN Headers
#include <linux/can.h>
#include <linux/can/raw.h>

#include <sys/socket.h>
#include <sys/ioctl.h>

#include <net/if.h>

// CAN IDs
const uint32_t SPEED_CAN_ID = 0x100;
const uint32_t RPM_CAN_ID   = 0x101;
const uint32_t TEMP_CAN_ID  = 0x102;

struct GPSData
{
```

```

    double latitude;
    double longitude;
};

struct VehicleData
{
    int speed;
    int rpm;
    int fuel;
    int engineTemp;
    GPSTData gps;
};

int main()
{
    srand(time(NULL));

    VehicleData vehicle;

    vehicle.speed      = 60;
    vehicle.rpm        = 2500;
    vehicle.fuel       = 100;
    vehicle.engineTemp = 90;

    vehicle.gps.latitude = 13.0827;
    vehicle.gps.longitude = 80.2707;

    // =====
    // MQTT Initialization
    // =====

    mosquitto_lib_init();

    struct mosquitto *mosq =
        mosquitto_new("TCU_Client", true, NULL);

```

```
if(!mosq)
{
    std::cerr << "Failed to create MQTT client\n";
    return 1;
}
```

```
if(mosquitto_connect(mosq,
                    "localhost",
                    1883,
                    60) != MOSQ_ERR_SUCCESS)
{
    std::cerr << "Failed to connect to broker\n";

    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();

    return 1;
}
```

```
std::cout << "Connected to MQTT Broker\n";
```

```
// =====  
// CAN Socket Initialization  
// =====
```

```
int can_socket;
```

```
struct sockaddr_can addr;  
struct ifreq ifr;
```

```
can_socket = socket(PF_CAN,
                    SOCK_RAW,
                    CAN_RAW);
```

```

if(can_socket < 0)
{
    perror("CAN Socket");
    return 1;
}

strcpy(ifr.ifr_name, "vcan0");

if(ioctl(can_socket,
        SIOCGIFINDEX,
        &ifr) < 0)
{
    perror("ioctl");
    close(can_socket);
    return 1;
}

addr.can_family = AF_CAN;
addr.can_ifindex = ifr.ifr_ifindex;

if(bind(can_socket,
        (struct sockaddr *)&addr,
        sizeof(addr)) < 0)
{
    perror("bind");
    close(can_socket);
    return 1;
}

std::cout << "Connected to vcan0\n";

// =====
// Main Loop
// =====

```

```
while(true)
{
    vehicle.speed = rand() % 120;
    vehicle.rpm = rand() % 6000;
    vehicle.fuel = rand() % 100;
    vehicle.engineTemp = 80 + rand() % 21;

    vehicle.gps.latitude += 0.0001;
    vehicle.gps.longitude += 0.0001;

    // =====
    // CAN Frames
    // =====

    struct can_frame speedFrame;
    struct can_frame rpmFrame;
    struct can_frame tempFrame;

    // Speed Frame

    speedFrame.can_id = SPEED_CAN_ID;
    speedFrame.can_dlc = 1;

    speedFrame.data[0] =
        vehicle.speed;

    // RPM Frame

    rpmFrame.can_id = RPM_CAN_ID;
    rpmFrame.can_dlc = 2;

    rpmFrame.data[0] =
        (vehicle.rpm >> 8) & 0xFF;

    rpmFrame.data[1] =
```



```
        vehicle.rpm & 0xFF;

// Temperature Frame

tempFrame.can_id = TEMP_CAN_ID;
tempFrame.can_dlc = 1;

tempFrame.data[0] =
    vehicle.engineTemp;

// =====
// CAN Transmission
// =====

if(write(can_socket,
        &speedFrame,
        sizeof(speedFrame)) < 0)
{
    perror("Speed CAN Write");
}

if(write(can_socket,
        &rpmFrame,
        sizeof(rpmFrame)) < 0)
{
    perror("RPM CAN Write");
}

if(write(can_socket,
        &tempFrame,
        sizeof(tempFrame)) < 0)
{
    perror("Temp CAN Write");
}
```

```
// =====  
// MQTT Messages  
// =====  
  
char locationMsg[100];  
char statusMsg[100];  
char diagnosticMsg[100];  
  
snprintf(locationMsg,  
          sizeof(locationMsg),  
          "Latitude=%.4f Longitude=%.4f",  
          vehicle.gps.latitude,  
          vehicle.gps.longitude);  
  
snprintf(statusMsg,  
          sizeof(statusMsg),  
          "Speed=%d Fuel=%d",  
          vehicle.speed,  
          vehicle.fuel);  
  
snprintf(diagnosticMsg,  
          sizeof(diagnosticMsg),  
          "RPM=%d EngineTemp=%d",  
          vehicle.rpm,  
          vehicle.engineTemp);  
  
// Publish Location  
  
mosquitto_publish(  
    mosq,  
    NULL,  
    "vehicle/location",  
    strlen(locationMsg),  
    locationMsg,  
    0,
```

```

        false);

// Publish Status

mosquitto_publish(
    mosq,
    NULL,
    "vehicle/status",
    strlen(statusMsg),
    statusMsg,
    0,
    false);

// Publish Diagnostics

mosquitto_publish(
    mosq,
    NULL,
    "vehicle/diagnostics",
    strlen(diagnosticMsg),
    diagnosticMsg,
    0,
    false);

// =====
// Console Output
// =====

std::cout << "\n----- Vehicle Data ----- \n";

std::cout << "Latitude    : "
            << vehicle.gps.latitude
            << std::endl;

std::cout << "Longitude   : "

```

```

        << vehicle.gps.longitude
        << std::endl;

    std::cout << "Speed      : "
        << vehicle.speed
        << " km/h"
        << std::endl;

    std::cout << "RPM        : "
        << vehicle.rpm
        << std::endl;

    std::cout << "Fuel         : "
        << vehicle.fuel
        << "%"
        << std::endl;

    std::cout << "Engine Temp : "
        << vehicle.engineTemp
        << " C"
        << std::endl;

    std::cout << "CAN TX -> "
        << "0x100(Speed) "
        << "0x101(RPM) "
        << "0x102(Temp)"
        << std::endl;

    sleep(1);
}

// Cleanup
close(can_socket);

mosquitto_disconnect(mosq);

```

```
    mosquitto_destroy(mosq);  
    mosquitto_lib_cleanup();  
  
    return 0;  
}
```

## Compile

None

```
g++ tcu_mqttCAN.cpp -o tcu_mqtt -lmosquitto
```

## Run

Terminal 1:

None

```
./tcu_mqttCAN
```

Terminal 2:

None

```
candump vcan0
```

Terminal 3:

None

```
mosquitto_sub -h localhost -t vehicle/status
```

Terminal 2 alone throwed an error :

```
aniruddhan@raspberrypi:~ $ candump vcan0  
SIOCGIFINDEX: No such device
```

## Why It Happened

A virtual CAN interface is not permanent.

When you reboot the Pi:

None

```
vcan0
```

is removed.

You must recreate it:

None

```
sudo modprobe vcan  
sudo ip link add dev vcan0 type vcan  
sudo ip link set up vcan0
```

every boot unless you automate it.

Solution for this ⇒ Create a startup script : **./start\_vcan.sh**

None

```
nano start_vcan.sh
```

Contents:

None

```
#!/bin/bash
```

```
sudo modprobe vcan  
sudo ip link add dev vcan0 type vcan 2>/dev/null  
sudo ip link set up vcan0
```

Make executable:

```
None  
chmod +x start_vcan.sh
```

This saves you from typing the commands manually.

Now to run the program

## Run

Terminal 1:

```
None  
./tcu_mqttCAN
```

**Terminal 2 :**

```
None  
./start_vcan.sh  
candump vcan0
```

```
aniruddhan@raspberrypi:~ $ ./start_vcan.sh
aniruddhan@raspberrypi:~ $ candump vcan0
vcan0 100 [1] 4F
vcan0 101 [2] 0F 74
vcan0 102 [1] 52
vcan0 100 [1] 38
vcan0 101 [2] 11 62
vcan0 102 [1] 5F
vcan0 100 [1] 35
vcan0 101 [2] 04 BD
vcan0 102 [1] 52
vcan0 100 [1] 48
vcan0 101 [2] 0F 9C
```

Terminal 3:

None

```
mosquitto_sub -h localhost -t vehicle/status
```



## The output explanation of terminal 2 :

candump output:

None

```
vcan0 100 [1] 4F
vcan0 101 [2] 0F 74
vcan0 102 [1] 52
```

decodes to:

| CAN ID | Raw Data | Decimal | Meaning            |
|--------|----------|---------|--------------------|
| 0x100  | 0x4F     | 79      | Speed = 79 km/h    |
| 0x101  | 0F 74    | 3956    | RPM = 3956         |
| 0x102  | 0x52     | 82      | Engine Temp = 82°C |

The next frame:

None

```
vcan0 100 [1] 38
vcan0 101 [2] 11 62
vcan0 102 [1] 5F
```

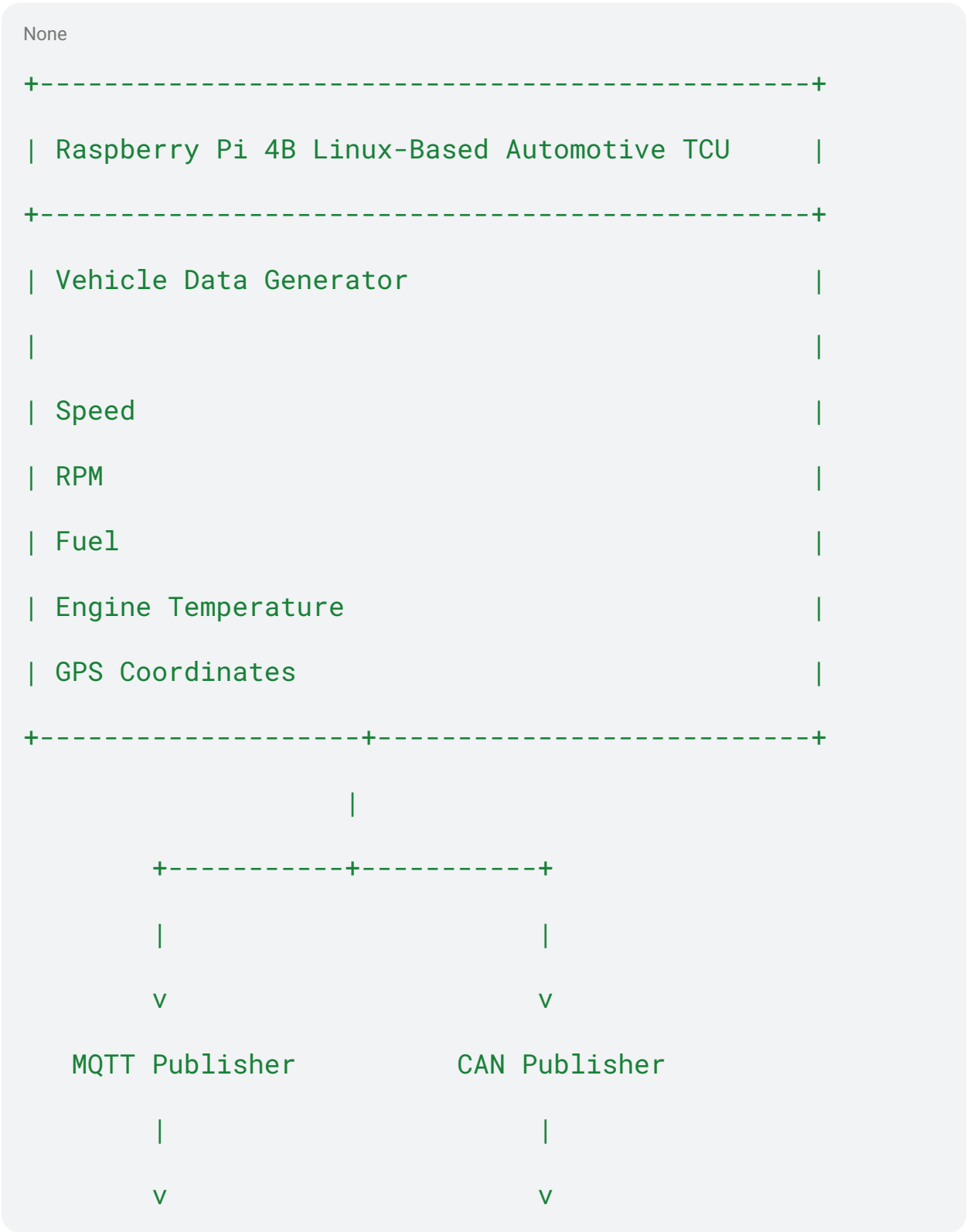
decodes to:

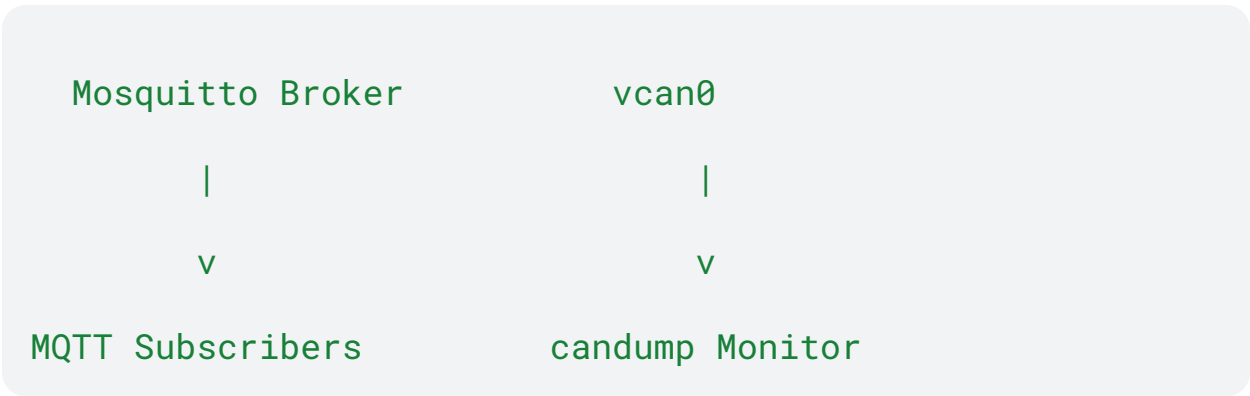
None

```
Speed      = 56 km/h
RPM        = 4450
Engine Temp = 95°C
```

which matches exactly what your simulator is transmitting.

# What You've Built So Far





# Project Milestones Achieved

| Module             | Status |
|--------------------|--------|
| Vehicle Simulation | ✓      |
| GPS Simulation     | ✓      |
| MQTT Broker Setup  | ✓      |
| MQTT Publishing    | ✓      |
| MQTT Subscribers   | ✓      |
| SocketCAN Setup    | ✓      |

CAN Frame  
Transmission 

CAN Frame  
Monitoring 

Linux Networking 

Telematics Data Flow 

=====

**one node transmits CAN frames and  
another node receives and decodes  
them.**

## Phase 4.1 Goal

Create:

None

`ecu_receiver.cpp`

that acts like a receiving ECU.

It will:

None

Listen on vcan0



Receive CAN Frames



Decode CAN IDs



Display Vehicle Signals

---

## Architecture

None

+-----+

| TCU Simulator |

| (Transmitter) |

+-----+

|

| CAN

v

vcan0

```
      |  
      v  
  
+-----+  
| ECU Receiver      |  
| (Receiver)        |  
+-----+
```

---

## ECU Receiver Code

Create:

```
None  
nano ecu_receiver.cpp
```

Paste:

```
None  
#include <iostream>  
  
#include <cstring>  
  
#include <unistd.h>  
  
#include <linux/can.h>
```

```
#include <linux/can/raw.h>
```

```
#include <sys/socket.h>
```

```
#include <sys/ioctl.h>
```

```
#include <net/if.h>
```

```
int main()
```

```
{
```

```
    int can_socket;
```

```
    struct sockaddr_can addr;
```

```
    struct ifreq ifr;
```

```
    struct can_frame frame;
```

```
    // Create CAN socket
```

```
    can_socket = socket(PF_CAN,
```

```
                        SOCK_RAW,
```

```
CAN_RAW);
```

```
if(can_socket < 0)
```

```
{
```

```
    perror("CAN Socket");
```

```
    return 1;
```

```
}
```

```
strcpy(ifr.ifr_name, "vcan0");
```

```
if(ioctl(can_socket,
```

```
        SIOCGIFINDEX,
```

```
        &ifr) < 0)
```

```
{
```

```
    perror("ioctl");
```

```
    close(can_socket);
```

```
    return 1;
```

```
}
```



```
addr.can_family = AF_CAN;

addr.can_ifindex = ifr.ifr_ifindex;


if(bind(can_socket,
        (struct sockaddr *)&addr,
        sizeof(addr)) < 0)
{
    perror("bind");
    close(can_socket);
    return 1;
}


std::cout << "ECU Receiver Listening on vcan0...\n";


while(true)
{
    int nbytes =
        read(can_socket,
            &frame,
```

```
        sizeof(frame));

if(nbytes < 0)
{
    perror("CAN Read");
    continue;
}

switch(frame.can_id)
{
    case 0x100:
    {
        int speed =
            frame.data[0];

        std::cout
            << "Received Speed      : "
            << speed
            << " km/h"
```

```
        << std::endl;

        break;
    }

    case 0x101:
    {
        int rpm =
            (frame.data[0] << 8) |
            frame.data[1];

        std::cout
            << "Received RPM          : "
            << rpm
            << std::endl;

        break;
    }
```

```
case 0x102:
{
    int temp =
        frame.data[0];

    std::cout
        << "Received Temperature : "
        << temp
        << " C"
        << std::endl;

    break;
}

default:
{
    std::cout
        << "Unknown CAN ID: 0x"
        << std::hex
```

```
        << frame.can_id  
        << std::dec  
        << std::endl;  
  
        break;  
    }  
}  
}  
}  
  
close(can_socket);  
  
return 0;  
}
```

---

## Compile

None

```
g++ ecu_receiver.cpp -o ecu_receiver
```

No extra libraries are needed.

---

# Run Test

## Terminal 1

Start vcan:

None

```
./start_vcan.sh
```

---

## Terminal 2

Start ECU receiver:

None

```
./ecu_receiver
```

Expected:

None

```
ECU Receiver Listening on vcan0...
```

---

## Terminal 3

Start TCU:

None

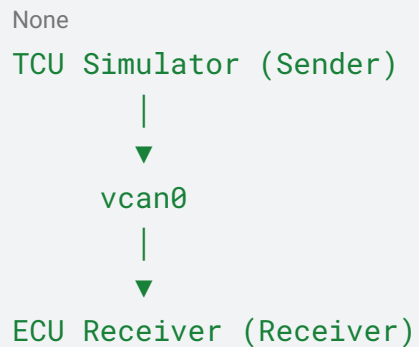
```
./tcu_mqttCAN
```

## Output in terminal 2 :

```
aniruddhan@raspberrypi:~ $ ./CAN_ecu_receiver
ECU Receiver Listening on vcan0...
Received Speed      : 57 km/h
Received RPM        : 5604
Received Temperature : 98 C
Received Speed      : 1 km/h
Received RPM        : 3508
Received Temperature : 88 C
Received Speed      : 1 km/h
Received RPM        : 4462
Received Temperature : 100 C
Received Speed      : 75 km/h
Received RPM        : 849
Received Temperature : 90 C
Received Speed      : 54 km/h
Received RPM        : 1666
Received Temperature : 85 C
Received Speed      : 30 km/h
Received RPM        : 2696
Received Temperature : 91 C
Received Speed      : 118 km/h
Received RPM        : 2810
Received Temperature : 100 C
Received Speed      : 19 km/h
```

This confirms that your complete CAN communication chain is working correctly.

You now have:



And the receiver is successfully decoding:

None

CAN ID 0x100 → Speed

CAN ID 0x101 → RPM

CAN ID 0x102 → Engine Temperature

For example:

None

Received Speed : 57 km/h

Received RPM : 5604

Received Temperature : 98 C

means:

None

0x100 -> 57

0x101 -> 5604

0x102 -> 98

which matches exactly what your TCU is transmitting